

A Comparison of Heuristic and Metaheuristic Methods for solving the Traveling Salesman Problem

1st Daniel Schafhäutle
Computational and Data Science
FHGR
Chur, Switzerland
daniel.schafhautle@stud.fhgr.ch

2nd Joshua Kohler
Computational and Data Science
FHGR
Chur, Switzerland
joshua.kohler@stud.fhgr.ch

3rd Gian-Luca Lanfranchi
Computational and Data Science
FHGR
Chur, Switzerland
gianluca.lanfranchi@stud.fhgr.ch

Abstract—The traveling salesman problem (TSP) is an NP hard combinatorial optimization problem, that asks for the shortest possible route that visits a set of cities and returns to the starting city. Solving TSPs exactly using brute-force methods quickly becomes computationally infeasible as the number of cities increases. However, there exists a number of heuristic and metaheuristic algorithms, that can find reasonably good solutions in a much shorter time. This paper presents a comparison of five such methods, namely: Simulated Annealing, Tabu Search, 2-Opt, Iterated Local Search and a Genetic Algorithm. Algorithms are evaluated using benchmark TSP instances of sizes $n = [10, 25, 50, 100, 200, 500]$ with $k = 30$ instances each. We perform this experiment with two types of instances: one with uniformly sampled cities and one with clustered cities. They are then compared based on the solution quality measured as the gap to the optimal solution obtained by the Concorde TSP solver, and the time required to find the solution. The goal of this paper is to compare the strengths and weaknesses of these algorithms, that are the basis of many state of the art methods for solving TSPs and provide a starting point for further research into this area.

I. Introduction

The traveling salesman problem (TSP) is a classic combinatorial optimization problem that asks for the shortest possible route that visits a set of cities and returns to the origin city. More formally, given a complete Graph $G = (V, E)$, where $V = \{v_1, v_2, \dots, v_n\}$ is the set of nodes representing the cities, and $d(v_i, v_j)$ is the distance associated with the edge between each pair of nodes $v_i, v_j \in V$. A solution to the TSP is an ordering of cities $(v_{\pi(1)}, v_{\pi(2)}, \dots, v_{\pi(n)})$ that minimizes the total tour length:

$$\left(\sum_{i=1}^{n-1} d(v_{\pi(i)}, v_{\pi(i+1)}) \right) + d(v_{\pi(n)}, v_{\pi(1)}) \quad (1)$$

Where $\pi(i)$ determines the index of the city that is visited at step i [1].

This is an NP-hard combinatorial optimization problem. That means, that there is no known polynomial time algorithm, that can solve it optimally for all instances [1]. It is so hard to solve, because the number of possible tours is $(n-1)!/2$ and grows with $\mathcal{O}(n!)$ [1]. This makes it infeasible to brute-force a solution for problems with more than a few nodes. A problem with 100 nodes has

10^{155} possible tours, to check all of them naively would take longer than the age of the universe. Because we don't have that kind of time, we need to find ways to quickly find solutions to TSPs that are good enough, meaning that they are not optimal, but sufficiently close to the optimal solution. However, for a lot of applications a solution that is not quite optimal is all that is needed. Heuristics are used to find these near optimal solutions within reasonable time.

II. Related Work

- Mention that exact solvers like Concorde use cutting-plane methods and branch-and-bound to find the true mathematical optimum.
- Briefly reference classic literature (like the Glover paper mentioned in your README) regarding how local search and metaheuristics revolutionized combinatorial optimization.

III. Materials and Methods

In this chapter several algorithms are introduced that are at the basis of many state of the art methods for solving the TSP. The goal is to introduce the reader to a variety of different techniques and concepts that are commonly used.

A. 2-Opt

2-Opt is one of the simplest and most widely used local search heuristics for the TSP [2]. Starting from some initial tour, it repeatedly removes two non-adjacent edges and reconnects the resulting two paths in the only other possible way, which is the same as reversing the segment between the two cut points. A move is accepted whenever it strictly decreases the tour length. The neighborhood of a tour with n cities therefore contains $\mathcal{O}(n^2)$ candidate moves. The search terminates in a local minimum, that is, a tour for which no single 2-Opt swap gives an improvement.

In this study, the initial tour is constructed by a deterministic nearest-neighbor heuristic (seeded for reproducibility): starting from a fixed city, the next city is always chosen as the closest unvisited one until all

cities are included. The 2-Opt search itself uses a first-improvement strategy: as soon as an improving swap (i, j) is found, it is applied immediately and the scan is restarted from the beginning of the tour. This is repeated until a full pass over all pairs (i, j) finds no improving move, at which point the tour is 2-optimal.

Algorithm 1 2-Opt

```

1:  $s = \text{GreedyInitialTour}()$ 
2:  $\text{improved} \leftarrow \text{true}$ 
3: while  $\text{improved}$  do
4:    $\text{improved} \leftarrow \text{false}$ 
5:   for  $i = 0$  to  $n - 3$  do
6:     for  $j = i + 2$  to  $n - 1$  do
7:        $a, b \leftarrow s_i, s_{i+1}$ 
8:        $c, d \leftarrow s_j, s_{(j+1) \bmod n}$ 
9:       if  $d(a, c) + d(b, d) < d(a, b) + d(c, d)$  then
10:        reverse  $s_{i+1 \dots j}$ 
11:         $\text{improved} \leftarrow \text{true}$ 
12:        break out of both loops
13:      end if
14:    end for
15:  end for
16: end while
17: return  $s$ 

```

The improvement condition compares the cost of the two edges that would be removed, $d(a, b) + d(c, d)$, with the cost of the two edges that would replace them, $d(a, c) + d(b, d)$. Because the segment between the two cut points is simply reversed, no other edge weights change and the cost difference can be evaluated in constant time per candidate move. 2-Opt is used here both as a single solver and as the embedded LocalSearch of the Iterated Local Search algorithm described below.

B. Simulated Annealing

Plain local search like 2-Opt always stops at the first local minimum it reaches, because it never accepts a move that makes the tour worse. Simulated Annealing fixes this by sometimes accepting worse tours on purpose, so that the search can escape a local minimum and keep looking for a better solution [2].

How willing the algorithm is to accept a worse tour is controlled by a single number called temperature T . At each step a neighbor tour is created with one random 2-Opt move, and the change in tour length $\Delta = \text{Cost}(\text{next}) - \text{Cost}(\text{current})$ is computed. A better tour ($\Delta < 0$) is always accepted. A worse tour is accepted only with probability $e^{-\Delta/T}$. When T is high this probability is close to one, so many worse tours are accepted. When T is low it is close to zero, so the algorithm behaves almost like normal 2-Opt local search.

The temperature is lowered after every step with a geometric cooling schedule, $T(t) = T_0 \cdot 0.9999^t$. The starting temperature T_0 is not fixed but estimated for each problem

so that the algorithm fits the scale of the distances: a few hundred random 2-Opt moves are sampled, the average of the worsening moves $\bar{\Delta}$ (mean delta) is taken, and T_0 is set so that about 60% of such moves would be accepted at the start, i.e. $T_0 = \bar{\Delta} / \ln(1/0.6)$. The search starts from a greedy nearest-neighbor tour and runs for a fixed budget of 100 000 iterations (or stops early once the temperature is practically zero), always returning the best tour seen so far.

Algorithm 2 Simulated Annealing

```

1:  $T_0 = \text{EstimateInitialTemperature}()$ 
2:  $\text{current} = \text{GreedyInitialTour}()$ 
3:  $\text{best} = \text{current}$ 
4: for  $t = 1$  to  $\text{MaxIterations}$  do
5:    $T = T_0 \cdot 0.9999^t$ 
6:   if  $T \approx 0$  then
7:     return  $\text{best}$ 
8:   end if
9:    $\text{next} = \text{Random2OptMove}(\text{current})$ 
10:   $\Delta = \text{Cost}(\text{next}) - \text{Cost}(\text{current})$ 
11:  if  $\Delta < 0$  then
12:     $\text{current} = \text{next}$ 
13:    if  $\text{Cost}(\text{current}) < \text{Cost}(\text{best})$  then
14:       $\text{best} = \text{current}$ 
15:    end if
16:  else
17:    accept  $\text{current} = \text{next}$  with probability  $e^{-\Delta/T}$ 
18:  end if
19: end for
20: return  $\text{best}$ 

```

A random 2-Opt move is the same edge swap as in normal 2-Opt, but here the two cut points are chosen at random instead of by scanning for an improvement. This keeps each step cheap while still letting the temperature decide whether a worse tour is kept or thrown away.

C. Tabu Search

D. Iterated Local Search

The idea of iterated local search is that instead of searching the whole space of all candidate solutions, we focus only on candidate solutions in the local neighborhood of an initial solution. These are usually generated by some kind of local search heuristic. It is conceptually simple, but has led to state-of-the-art algorithms for many problems [3]. In its essence, iterated local search builds a chain of solutions by exploring the currently best solutions local neighborhood but introduces some randomness in order to not get stuck in local minima.

Algorithm 3 Iterated Local Search

```
1:  $s_0 = \text{GenerateInitialSolution}()$ 
2:  $s^* = \text{LocalSearch}(s_0)$ 
3:  $s_{\text{best}} = s^*$ 
4: repeat
5:    $s' = \text{Perturbation}(s^*)$ 
6:    $s^{*'} = \text{LocalSearch}(s')$ 
7:   if  $\text{Cost}(s^{*'}) < \text{Cost}(s_{\text{best}})$  then
8:      $s_{\text{best}} = s^{*'}$ 
9:   end if
10:  if  $\text{Cost}(s^{*'}) < \text{Cost}(s^*)$  then
11:     $s^* = s^{*'}$ 
12:  end if
13: until termination condition met (e.g., iterations = 200)
14: return  $s_{\text{best}}$ 
```

Iterated local search can be used with any heuristic optimization algorithm (LocalSearch) that takes in a tour and returns one that is at least as good as the input. It starts from some kind of initial solution s_0 that is improved to s^* by one iteration of the LocalSearch algorithm. This is the current solution. Then we try to improve s^* for a number of iterations or until a termination condition is met.

For the algorithm to be able to escape the local neighborhood, instead of applying LocalSearch directly to the current solution s^* again, first a change or Perturbation is applied that leads to an intermediate solution s' . Then LocalSearch is applied to s' and we get a new candidate solution $s^{*'}$. If this candidate solution $s^{*'}$ has a better Cost than the current one, it becomes the new current solution $s^{*'}$. If it is better than the current best, it also becomes the new current best s_{best} .

The Cost is the function that is to be minimized by the algorithm. In the case of the TSP that is the length of the tour.

The Perturbation is the mechanism that allows the algorithm to escape local minima. This function should not be reversible by the LocalSearch. So if the algorithm steps from $s_1^* \rightarrow s_2^*$ it then should be unable to do $s_2^* \rightarrow s_1^*$ in the same iteration.

Usual choices, and what was also used in this study are 2-Opt for the LocalSearch and 4-Opt for the Perturbation, this is a good choice because a 4-Opt move cannot be undone by a 2-Opt move and thereby return to the last starting point of the current iteration.

4-Opt (also called double-bridge move) randomly splits the tour into four segments $A + B + C + D$ and reconnects them as $A + C + B + D$ [3].

E. Genetic Algorithm

IV. Experiments and Results

Detail exactly how you conducted the experiment so someone else could replicate it

A. Setup

B. Dataset Generation

To evaluate the performance of the TSP algorithms a synthetic benchmark dataset of Euclidean TSP instances is generated. The generation process is controlled by a Python script that creates two types of point distributions: uniform and clustered. The script uses a fixed random seed (42) to ensure reproducibility.

1) Instance sizes and repetitions: For each problem size $n \in \{10, 25, 50, 100, 200, 500\}$ 30 independent instances were generated. The same set of instances is used for all algorithms, allowing a fair comparison.

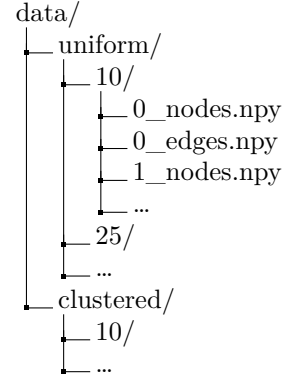
2) Uniform instances: Uniform instances are created by drawing n coordinates independently from a uniform distribution over the unit square $[0, 1]^2$. The Euclidean distance between every pair of coordinates is computed to form a symmetric distance matrix.

3) Clustered instances: Clustered instances are designed to mimic real-world scenarios where coordinates of e.g. cities or harbors form clusters. The number of clusters is set to $\max(1, n/10)$. Cluster centers are sampled uniformly from $[0.1, 0.9]^2$ (avoiding the extreme borders). Each node is assigned to a center in a round-robin fashion and its coordinates are drawn from a normal distribution around that center with a standard deviation of 0.05. The resulting coordinates are clipped to the unit square to keep all points inside $[0, 1]^2$. The distance matrix is then computed as for the uniform case.

4) Storage format: Each instance is stored in two NumPy binary files (“.npz”):

- {id}_nodes.npz: an $n \times 2$ array of node coordinates.
- {id}_edges.npz: an $n \times n$ matrix of pairwise Euclidean distances.

The directory structure is:



5) Test data option: The script also provides a `-testdata` flag that writes the generated instances into a separate `data/test/` directory. This can be used for hyperparameter tuning or preliminary experiments without interfering with the main benchmark dataset.

All dataset generation parameters (random seed, cluster spread, number of repetitions) are kept constant throughout the experiments to ensure a consistent basis for comparison.

- 6) Benchmark: Mention greedy baseline
- 7) Hardware / Environment:

C. Results

- Performance Metrics
- Plots

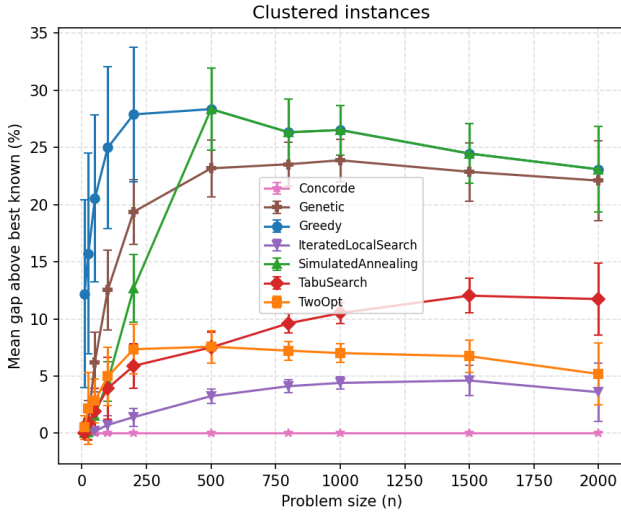


Fig. 1. Optimality gap of clustered algorithms for different sizes and distributions

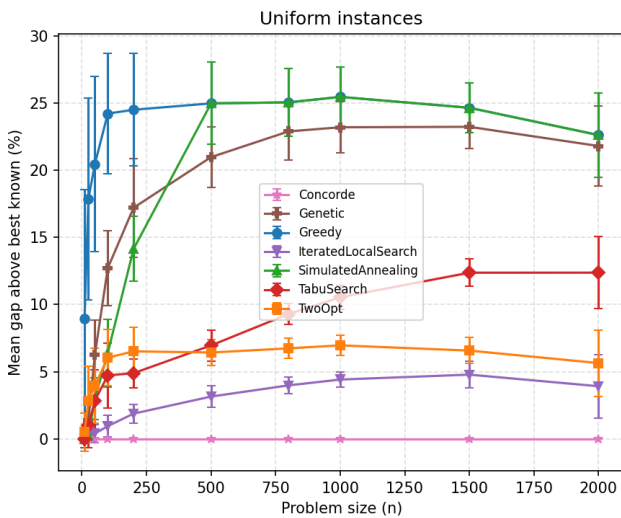


Fig. 2. Optimality gap of uniformed algorithms for different sizes and distributions

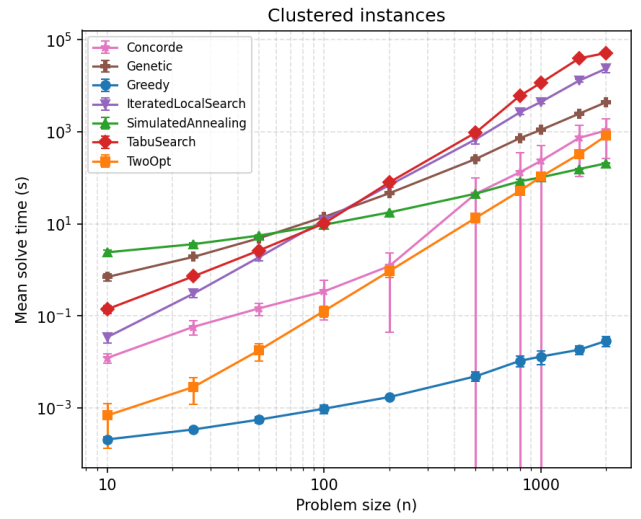


Fig. 3. Time scaling of clustered algorithms for different sizes and distributions.

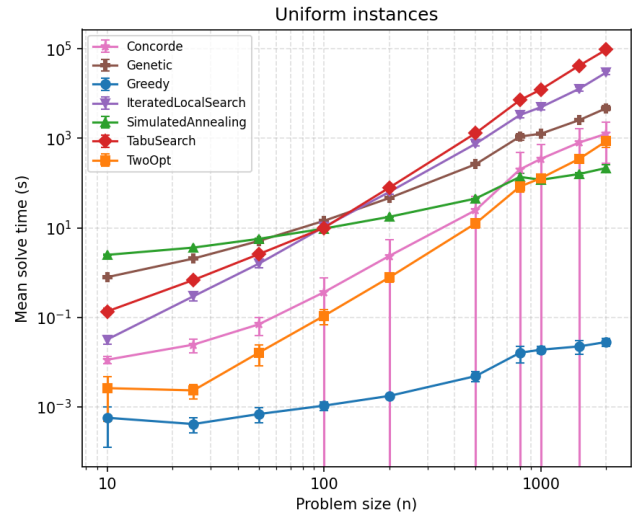


Fig. 4. Time scaling of uniformed algorithms for different sizes and distributions.

V. Discussion

Trade off analysis. Speed vs Quality.

Best all rounder, fast but sloppy, accurate but expensive...

Explain why the results look like that. Why is one algorithm better able to handle clusters than another. etc.

VI. Conclusion

Summarize core finding.

suggest future directions

- Other algorithms. Hyperparameter tuning.

References

- [1] M. R. Garey and D. S. Johnson, Computers and Intractability. wh freeman New York, 2002, vol. 29.

- [2] C. Rego and F. Glover, “Local Search and Metaheuristics for the Traveling Salesman Problem.”
- [3] H. R. Lourenço, O. C. Martin, and T. Stützle, “Iterated Local Search: Framework and Applications,” in Handbook of Metaheuristics, M. Gendreau and J.-Y. Potvin, Eds. Cham: Springer International Publishing, 2019, pp. 129–168.

VII. The First Appendix

A. Independence Declaration

if needed for the papper

We hereby declare that we have written this thesis independently as a group, that we have not used any sources or aids other than those indicated, and that we have not submitted this work, in whole or in part, for any other examination.

All content taken verbatim or in substance from published or unpublished sources has been clearly identified as such. Any use of AI-based tools has been disclosed in accordance with the institution’s guidelines for scientific writing.

We have followed the rules set out in the Guidelines for Scientific Writing. We understand that any breach of this declaration may lead to the withdrawal of the qualifications or academic title awarded on the basis of this thesis.

Chur, June 5, 2026 Chur, June 5, 2026 Chur, June 5, 2026

Signature

Signature

Signature

AI Assistance Declaration

Claude (Anthropic) and Codex were used during the writing of this report, mainly for translating technical content from German to English and occasionally as a sounding board when working through how to present an argument. Every output was checked and reworked by the authors before being included.